HermSec-MoA: Comparing Scanner-Aided, Single-Agent, Multi-Agent, and Hybrid Repository Security Review

Seth Whenton
University of Passau
Passau, Germany

Ghazal Pouresfandiyar Borojeni
University of Passau
Passau, Germany

Abstract

HermSec is a desktop app that scans local code repositories for security issues. The main goal is simple. A user should pick a project, press scan, and get a clear report. HermSec first checks the project type, chooses useful scanners, runs the tools, and writes a report. The app also supports model-aided review.

This paper reports completed tests on our Ghazal dataset of 20 projects and 184 labelled findings. The tested configurations were Deep assisted scan, Single Agent inspection, MoA Low, MoA High, Scanner + MoA Low, and Scanner + MoA High. These tests compare scanner-backed review, direct agent review, multi-agent review, and a hybrid scanner-agent review.

The best F1 score was Scanner + MoA High with 0.7640. Single Agent reached 0.6462 and was faster. MoA High reached 0.5455, MoA Low reached 0.5433, Scanner + MoA Low reached 0.5333, and Deep assisted reached 0.4381. The result shows that the hybrid mode gave the strongest balance. The current HermSec software uses candidate-first tasks, bounded file tools, evidence checks, sanitizer-aware filtering, and efficient non-US models to make these modes more reliable.

Keywords

static analysis, software security, repository scanning, multi-agent systems, vulnerability detection, code review

1 Introduction

Developers need security feedback while they work. Static application security testing tools help because they can check source code without running the program [3, 8]. These tools are useful, but they can also miss bugs or report too many false alarms [22].

HermSec was built as a simple desktop security scanner for local projects. The user selects a folder and asks HermSec to scan it. HermSec inspects the repository, detects languages and project files, chooses the scanners that fit the project, runs the scan, and writes a report. This matters because a JavaScript project, Java project, Python project, Rust project, and PHP project do not need the same scanner set.

The project also studies model-aided security review. We wanted to know if agent inspection can improve the normal scanner workflow. We also wanted to test if a mixture of agents can do better than a single model. Finally, we tested a hybrid mode where scanners and MoA agents run separately, then a judge and aggregator merge the findings.

The research question is:

> Can HermSec improve repository security review by combining static scanners, single-agent inspection, MoA inspection, and Scanner + MoA aggregation while using efficient low-cost models?

This paper explains the current HermSec product, the scan modes, the dataset, the results, and the next work needed.

2 HermSec Product Overview

HermSec is not only a command line test tool. It is a desktop app with a chat-style workflow. The app has project selection, scan mode selection, live progress cards, scanner settings, model provider settings, Doctor checks, automation settings, and report links.

The most important product idea is the scan flow. HermSec first inspects the project. It checks files, languages, package manifests, lockfiles, and configuration files. It then chooses the scanners that match the project. For example, a Node.js project can use JavaScript rules, dependency checks, and secret scanning. A Python project can use Python rules, Bandit, pip-audit style checks, and dependency intelligence. A Java project can use Java rules and Java taint heuristics. This adaptive choice helps avoid installing or running every scanner for every project.

HermSec currently supports built-in heuristics and external scanners such as Semgrep, Gitleaks, Trivy, Bandit, OSV-Scanner, and Checkov [1, 2, 7, 15, 16, 18]. The app can show which scanners are installed, missing, enabled, and useful for the current project.

Figure 1 shows the scan workflow. The user chooses a scan mode, then HermSec keeps the progress card in the chat and writes the report when the scan finishes.

Figure 2 shows the settings pages for agent and provider setup. Figure 3 shows scanner management.

3 Related Work

Static analysis has been used for many years to detect security problems before code runs [3, 8]. It is still important because it is fast and can run in development. However, static tools need careful rules and testing. Benchmarks and labelled examples help show where a tool is strong or weak [22].

Modern scanners often focus on different areas. Semgrep is useful for rule-based source scanning. Gitleaks is used for secrets. Trivy and OSV-Scanner support dependency and advisory checks. Bandit is used for Python security linting [1, 7, 15, 16, 18]. HermSec uses this multi-tool idea and normalizes findings into one report.

Large language models can help explain code, but they can also make false claims [14]. Agent patterns such as ReAct show how a model can use tools step by step [21]. Multi-agent debate and mixture-of-agents work also show that several model answers can improve some tasks [6, 17]. HermSec uses a similar idea for code review. The MoA settings were also inspired by configurable agent panels in Hermes Agent [11].

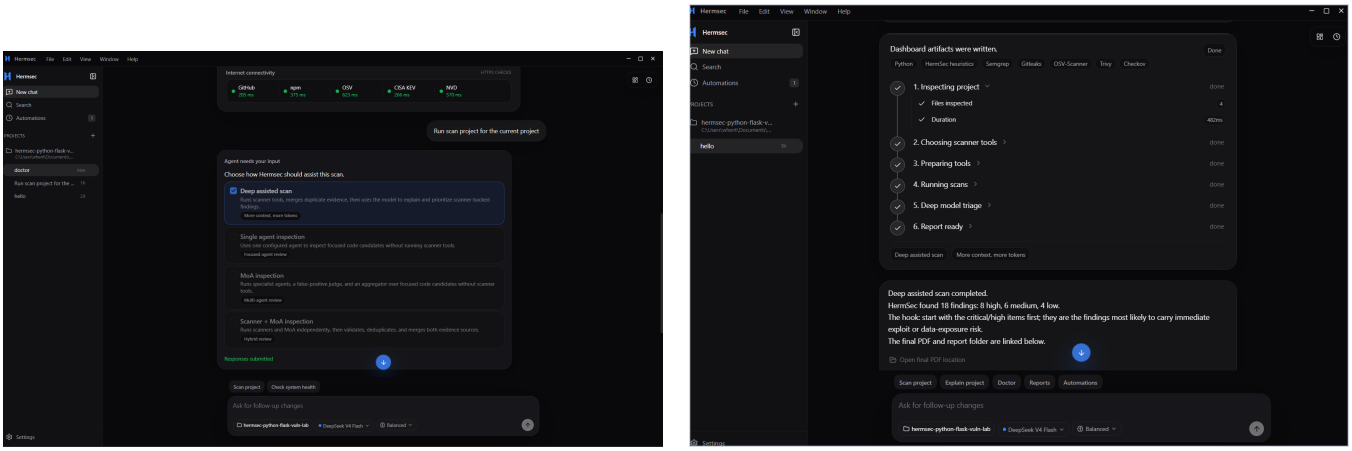

Figure 1: HermSec scan workflow. The app asks the user to choose a mode, shows progress, and then shows a short report summary.

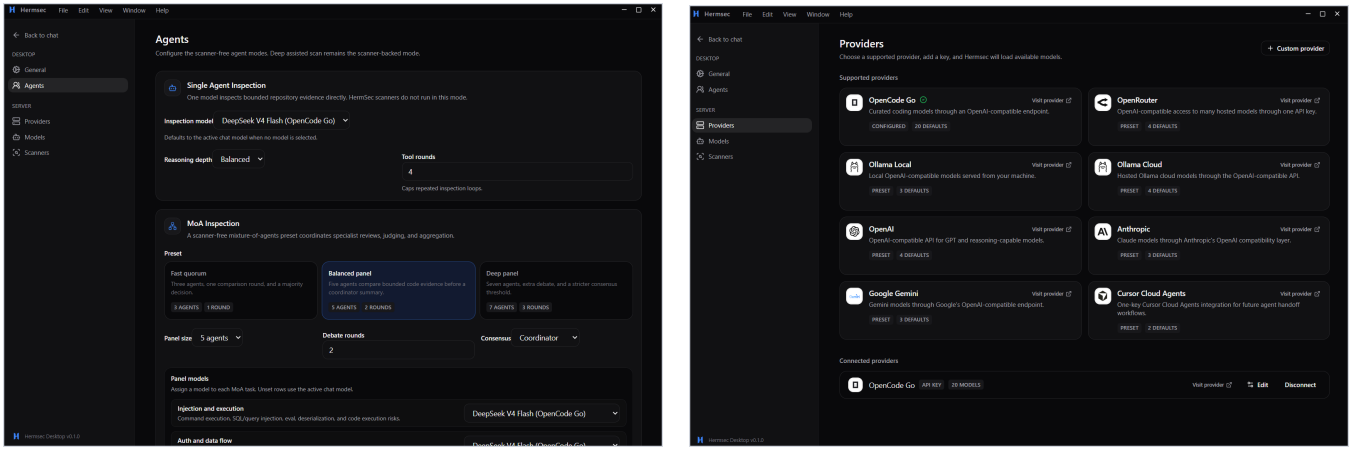

Figure 2: HermSec settings. Left: Single Agent and MoA settings. Right: model provider setup.

4 Current HermSec Techniques

HermSec has four visible scan modes. MoA and Scanner + MoA have Low and High panel settings in the app. This paper records the latest Ghazal results for Deep assisted, Single Agent, MoA Low, MoA High, Scanner + MoA Low, and Scanner + MoA High.

4.1 Deep Assisted Scan

Deep assisted scan is scanner-backed. HermSec inspects the repository, chooses scanner tools, prepares them, runs them, and normalizes the findings. A model then explains scanner-backed evidence. This mode is closest to the original product idea. It gives broad tool coverage and then uses a model for summary and explanation. The current CLI also chunks model explanation work and uses watchdogs so the model step cannot block the scan forever.

4.2 Single Agent Inspection

Single Agent inspection does not run scanners. HermSec first creates focused investigation tasks from deterministic candidate discovery. These candidates cover source-to-sink patterns such as SQL injection, command execution, path traversal, XSS, hardcoded secrets, redirects, dependency risk, and configuration risk. The model receives bounded repository evidence using safe read-only tools: list files, search code, and read snippets. The model cannot execute shell commands, install packages, or read outside the selected repository. After the model proposes findings, HermSec checks that files and snippets really exist before adding findings to the report.

4.3 MoA Inspection

MoA inspection uses several specialist agents. The Low panel uses three specialists: injection and execution, auth and data flow, and secrets and config. The High panel uses five specialists by adding database and storage, and config and IaC. A false-positive judge

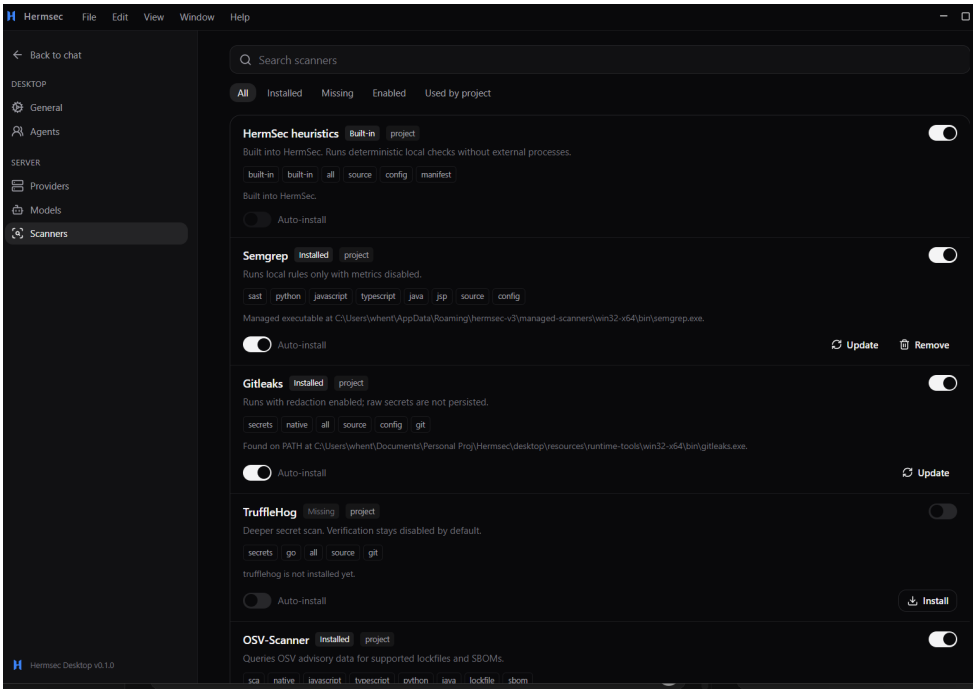

Figure 3: Scanner settings show installed, missing, enabled, and project-relevant tools.

checks the candidate findings. An aggregator then writes the final accepted findings.

This design improves simple broad prompting because agents do not just receive ”inspect this repo”. They receive focused tasks, such as checking an injection candidate, a hardcoded secret candidate, or a sanitizer claim. This should reduce token waste because each agent reads selected evidence instead of reading the whole project. The judge rejects unsupported model claims instead of keeping them for human review.

4.4 Scanner + MoA Inspection

Scanner + MoA is the hybrid mode. Scanners run normally and produce scanner findings. MoA agents also inspect the project separately. The judge receives both scanner candidates and agent candidates. The aggregator deduplicates and writes the final findings.

This mode tries to get the coverage advantage of scanners and the reasoning advantage of agents. It is also the most expensive mode because it runs both scanner and agent work. The current design adds final evidence validation so unsupported model claims are rejected. The latest CLI also adds sanitizer-aware filtering for safe patterns such as prepared SQL statements, HTML escaping, canonical path checks, `basename`, and `filepath.Clean`. This reduced clean-project false positives in the final Scanner + MoA High run.

4.5 Efficient Model Routing

The experiment used OpenCode Go as the provider route [12]. Deep assisted and Single Agent used `deepseek-v4-flash`. MoA and Scanner + MoA used `deepseek-v4-flash`, `mimo-v2.5`, `deepseek-v4-pro`, and `minimax-m3`. We avoided US flagship models in the experiment table.

This model choice was made for cost and speed reasons. DeepSeek documents context caching and lower cached-token pricing [4, 5]. MiMo V2.5 is described as an agent-capable model family with inference optimization work [19, 20]. MiniMax-M3 is presented as a coding and agentic model [9]. We did not measure exact money cost in this run, so cost claims are design claims, not final cost measurements.

5 Harness Flow

Every HermSec scan starts with repository inspection. Then the selected mode runs. After the mode finishes, HermSec validates evidence, deduplicates findings, normalizes them, and writes HTML, JSON, and PDF-ready report artifacts.

6 Method

6.1 Dataset

We tested HermSec on our dataset of 20 local projects. The dataset contains 184 labelled findings. It includes vulnerable and clean projects across JavaScript, TypeScript, Python, Java, PHP, Ruby, Go, Rust, C/C++, C#, Docker, and GitHub Actions style configuration. The findings cover injection, command execution, secrets, authentication, dependency risk, database risk, API risk, crypto issues, and configuration problems. CWE labels were used where possible [10].

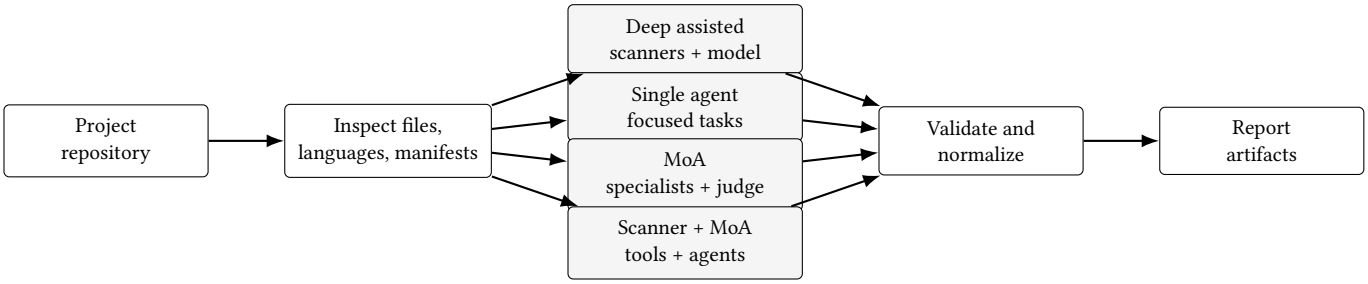

Figure 4: Current HermSec harness flow. The selected mode changes how findings are created, but all final findings pass through validation and reporting.

The risk types also match common web security issues described by OWASP [13].

6.2 Modes Tested

The final metric table uses the completed full-dataset runs. The tested configurations were:

- Deep assisted scan with `deepseek-v4-flash`.
- Single Agent with `deepseek-v4-flash`.
- MoA Low with three specialists, a judge, and an aggregator.
- MoA High with five specialists, a judge, and an aggregator.
- Scanner + MoA Low with scanners and the low MoA panel.
- Scanner + MoA High with scanners and the high MoA panel.

The MoA and Scanner + MoA model pool was `deepseek-v4-flash`, `mimo-v2.5`, `deepseek-v4-pro`, and `minimax-m3`. All model calls used OpenCode Go. Scanner + MoA Low completed 17 of 20 runs, so it is included as a secondary row. Scanner + MoA High completed all 20 runs and is the main hybrid result.

6.3 Metrics

We used true positives, false positives, false negatives, precision, recall, and F1. Precision means the share of reported findings that were correct. Recall means the share of labelled findings that were found. F1 is the balance between precision and recall.

Each main configuration was run against the 20 projects. Scanner + MoA Low completed 17 successful runs; the other rows completed 20 successful runs. The raw comparison note is saved at `results/ghazal-main-iteration6-final-score-summary-20260629.md`.

7 Results

Table 1 gives the main result. Figure 5 shows precision, recall, and F1. Figure 6 shows the true-positive and false-positive output count.

8 Discussion

The result shows that the working method improved the balance between coverage and false positives. Deep assisted found 85 true positives, but it still produced 119 false positives. This is expected because it depends strongly on scanner output.

Single Agent performed well for a cheaper mode. It found 126 true positives with 80 false positives and reached an F1 score of 0.6462. This is important because it shows that focused candidate tasks and evidence checks can make one model useful without running scanners.

MoA Low and MoA High were more balanced than Deep assisted, but they did not beat Single Agent. MoA Low reached an F1 score of 0.5433. MoA High reached 0.5455. This shows that adding more agents is not automatically better. The agents need good focused tasks and good rejection rules.

Scanner + MoA High gave the best result. It found 157 true positives, had 70 false positives, and reached an F1 score of 0.7640. Scanner + MoA Low reached 0.5333, but only 17 of 20 runs completed successfully. This supports the main idea of HermSec. Scanners are useful for coverage, agents are useful for reasoning, and a judge/aggregator can merge the evidence into a stronger report.

The current candidate-first technique is useful because it avoids asking every agent to read the whole project. HermSec first builds focused tasks and then sends those tasks to the agents. The latest tuning also added sanitizer-aware filtering, so safe code such as escaped HTML, prepared SQL, canonical path checks, and cleaned paths is less likely to be reported. This reduces wasted context and helps the judge reject unsupported claims. The full run also showed a practical tradeoff. Scanner + MoA High had the best score, but Single Agent was much faster.

9 Problems Faced

The main problem during testing was model reliability. Some model calls returned empty content or timed out. HermSec was updated with watchdogs, shorter retries, and fallback handling so a scan would not loop forever.

Another problem was scanner noise. Scanner-backed modes can find many real issues, but they can also report too much. Clean projects were useful because they showed where scanner or hybrid modes were too noisy.

A third problem was evidence safety. Model findings can mention files or lines that do not exist. HermSec now revalidates evidence before final reporting. Unsupported model claims are rejected instead of being shown as real vulnerabilities.

A final issue was runtime. Full MoA-style runs were slower than Single Agent. This is why the product needs checkpoints, caching, and focused tasks. These features help keep the app usable even when the agent panel is larger.

Table 1: Main results from completed full runs on the Ghazal dataset.

Mode	Runs	OK	TP	FP	FN	Precision	Recall	F1	Time (s)
Deep assisted	20	20	85	119	99	0.4167	0.4620	0.4381	715
Single Agent	20	20	126	80	58	0.6117	0.6848	0.6462	269
MoA Low	20	20	91	60	93	0.6026	0.4946	0.5433	1178
MoA High	20	20	84	40	100	0.6774	0.4565	0.5455	1560
Scanner + MoA Low	20	17	88	58	96	0.6027	0.4783	0.5333	4416
Scanner + MoA High	20	20	157	70	27	0.6916	0.8533	0.7640	2321

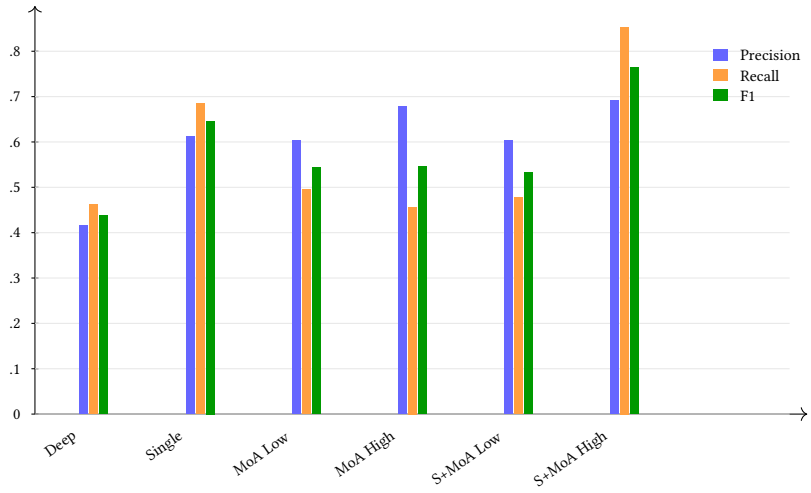

Figure 5: Precision, recall, and F1 for the completed full Ghazal runs. Scanner + MoA High had the best F1.

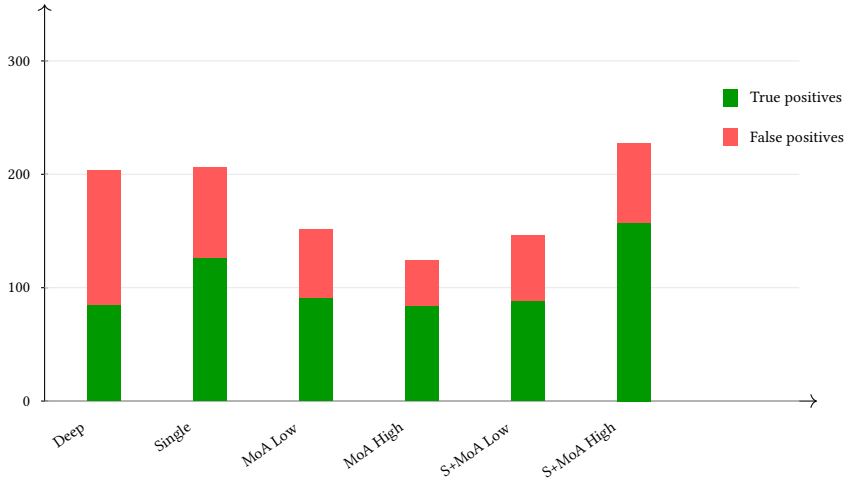

Figure 6: True-positive and false-positive counts. Scanner + MoA High found the strongest balance on this dataset.

10 Threats to Validity

The dataset is useful, but it is still limited. It has 20 local projects and 184 labelled findings. The results may change on larger real repositories. The scoring is also based on matching findings to known labels, so matching rules can affect precision and recall.

The experiment used efficient models through OpenCode Go. Different model providers, different model versions, or different temperature settings may change the result. Exact token cost was not measured, so this paper only reports runtime and finding metrics.

11 Future Work

Future work should make HermSec more automatic and easier to use. The long-term product goal is a much smaller app with one main scan button. The app should inspect the project, choose tools, run the right mode, and write the report with less setup.

HermSec should also support email reporting. The agent could connect to a mailbox and send scan reports after new scans. This would help scheduled security checks.

Another useful future feature is an MCP server for HermSec. This would let agents inside editing tools call the HermSec CLI directly. For example, an editor agent could ask HermSec to scan the current project, wait for the result, and open the report without the user switching tools.

Another important feature is change detection. If a project has not changed since the last scan, HermSec should know that and avoid running the same full scan again. It could compare file hashes, git commits, package lockfiles, and stored report metadata. This would save time and reduce model tokens.

The hybrid mode also needs better runtime and cost control. Scanner + MoA High gave the best result, but it took a long time. Better scanner candidate limits, stronger judge prompts, better evidence checks, caching, and cost tracking should improve it.

12 Conclusion

HermSec is a desktop repository security scanner with scanner-backed and agent-backed review modes. The completed Ghazal test used 20 projects and 184 labelled findings. Scanner + MoA High gave the best F1 score with 0.7640. Single Agent was the fastest strong mode with an F1 score of 0.6462.

The main lesson is simple. Scanners are useful for coverage. Agents are useful for focused reasoning and explanation. The best direction for HermSec is not only scanners or only agents. The best direction is a careful hybrid design with focused tasks, strong evidence validation, and better false-positive control.

References

[1] Aqua Security. 2026. Trivy: Vulnerability, Misconfiguration, Secret, and SBOM Scanner. https://github.com/aquasecurity/trivy. Accessed: 2026-06-29.

[2] Bridgecrew. 2026. Checkov: Static Code Analysis for Infrastructure as Code. https://github.com/bridgecrewio/checkov. Accessed: 2026-06-29.

[3] Brian Chess and Gary McGraw. 2004. Static Analysis for Security. *IEEE Security & Privacy* 2, 6 (2004), 76–79.

[4] DeepSeek. 2026. API Pricing. https://api-docs.deepseek.com/quick_start/pricing. Accessed: 2026-06-29.

[5] DeepSeek. 2026. Context Caching. https://api-docs.deepseek.com/guides/kv_cache. Accessed: 2026-06-29.

[6] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. https://arxiv.org/abs/2305.14325.

[7] Google OSV. 2026. OSV-Scanner. https://github.com/google/osv-scanner. Accessed: 2026-06-29.

[8] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*.

[9] MiniMax. 2026. MiniMax-M3. https://www.minimax.io/models/text/m3. Accessed: 2026-06-29.

[10] MITRE. 2026. Common Weakness Enumeration. https://cwe.mitre.org/. Accessed: 2026-06-29.

[11] Nous Research. 2026. Hermes Agent. https://github.com/nousresearch/hermes-agent. Accessed: 2026-06-29.

[12] OpenCode. 2026. OpenCode Go API. https://opencode.ai/zen/go/v1. Accessed: 2026-06-29.

[13] OWASP Foundation. 2021. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/. Accessed: 2026-06-29.

[14] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*.

[15] PyCQA. 2026. Bandit: Security Linter from PyCQA. https://bandit.readthedocs.io/. Accessed: 2026-06-29.

[16] Semgrep, Inc. 2026. Semgrep: Lightweight Static Analysis for Many Languages. https://semgrep.dev/. Accessed: 2026-06-29.

[17] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. https://arxiv.org/abs/2406.04692.

[18] Zachary Wilson. 2026. Gitleaks: Detect and Prevent Secrets in Git Repositories. https://github.com/gitleaks/gitleaks. Accessed: 2026-06-29.

[19] Xiaomi. 2026. Model Release: MiMo-V2 Series. https://mimo.mi.com/docs/en-US/updates/model. Accessed: 2026-06-29.

[20] XiaomiMiMo. 2026. Full-Pipeline Inference Optimization for MiMo-V2.5 Series. https://mimo.xiaomi.com/blog/mimo-v2-5-inference. Accessed: 2026-06-29.

[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629.

[22] Misha Zitser, Richard Lippmann, and Tim Leek. 2004. Testing Static Analysis Tools Using Exploitable Buffer Overflows from Open Source Code. In *Proceedings of the 12th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 97–106.